

HelixCode — Authentication / Login Guide (real-account, SSH-key)

Revision: 4 **Last modified:** 2026-07-01T00:00:00Z

Status: IN PROGRESS — being built. This guide describes the *intended* behavior of the real-account + **SSH-key challenge-response** authentication model per the approved pivot spec ([docs/superpowers/specs/2026-07-01-auth-pivot-ssh-key.md](https://docs.superpowers/specs/2026-07-01-auth-pivot-ssh-key.md), which supersedes the earlier PAM-login part of the original design). The components (the helix-auth gate, the host-native code-server unit, the Caddy forward_auth render) are under active development and **not yet validated**; nothing below should be read as “already working / already tested.” Feature ledger with the live validation verdicts: docs/features/Status.md.

How a HelixCode session is tied to a **real host user account**, how the login screen verifies that account by a **signed SSH-key challenge** (no password, nothing stored), and how the editor Explorer defaults to your projects folder (a convenience view, not a security boundary) while the whole session keeps your full real-user capability.

1. The model — session runs AS the real user

A HelixCode session is bound to a **real host user account** (default milosvasic). code-server runs **host-native** as that user (a systemd --user service), so the editor session **natively inherits the real user’s identity and environment** — nothing is copied, synced, or re-created:

- **SSH keys** — the host ~/.ssh is the user’s own directory. Every key already registered with our Git services works for git over SSH **from the editor’s integrated terminal** (e.g. git@github.com:...), with no extra setup.
- **.bashrc / profile** — the integrated terminal spawns a **login shell** (bash -l), so ~/.bashrc and the login profile are sourced:

exported vars, PATH additions, and init steps are all present in a fresh terminal.

- **All host binaries / utilities** — because the process runs as the real user on the host, every system app / binary / tool the user normally has is available in the terminal.

This is why the runtime is **host-native as the real user** rather than containerized: it is the clean way to give a session *all* the host binaries and the real identity the user usually has. The **Caddy TLS edge stays containerized** (rootless Podman, §11.4.76 / §11.4.161, unchanged from the TLS release); only the editor + auth processes are host-native, because they fundamentally require the host's real identity and toolchain.

```
Browser —HTTPS→ Caddy (TLS edge, containerized)
                    |— forward_auth → helix-auth (host-native,
loopback, NON-root)
                    |— reverse_proxy → code-server (host-native
AS milosvasic)
                                inherits ~/.ssh,
~/.bashrc, all host binaries
                                workspace =
$PROJECTS_ROOT (Explorer default view, NOT a jail)
                                terminal + Open Folder +
extensions = full real-user host access
```

2. Login — SSH-key challenge-response, verified against your own authorized keys

The login screen does **not** ask for a password. Instead it proves you control one of the real account's SSH keys, by asking you to **sign a fresh server-issued challenge**. A small forward-auth gate — **helix-auth** — that sits behind Caddy runs the challenge/verify flow **as the non-root real user** and stores no secret:

1. You open `https://<host>:<port>` and are sent to the HelixCode **login page**. The page shows a **fresh challenge** — a random nonce the gate signed (bound to a short TTL so it cannot be forged or replayed) — together with the exact command to run locally.
2. In a local terminal (on the machine that holds your private key) you sign the challenge with `ssh-keygen`:

```
printf %s '<challenge>' | ssh-keygen -Y sign -n helixcode-
login -f ~/.ssh/id_ed25519
```

This produces an SSH signature (the `helixcode-login` namespace scopes it to this login flow). Your **private key never leaves your machine**.

3. You **paste the signature** back into the login page and submit (the form posts { `principal`, `signature` } to `POST /login`).
4. `helix-auth` verifies it: first that the challenge it issued is still valid and unexpired (server-issued, not replayed), then that the signature checks out against the real account's keys —

```
ssh-keygen -Y verify -n helixcode-login -f <allowed_signers>
-I <principal> -s <sigfile>
```

— where `<allowed_signers>` is derived from **milosvasic's** `~/.ssh/authorized_keys` (each authorized key becomes an allowed-signers line for the configured principal). On success the gate issues a **signed session cookie** and Caddy proxies you to the editor. On failure, access is **denied** (401, generic error, rate-limited) — Caddy never proxies to code-server.

Nothing is stored — anywhere, and there is NO password.

There is no password parameter and no stored password: not in the repo, not in config, not in an env var, not in logs, and not in captured evidence. The gate reads only the account's **authorized_keys** (public material) to verify a signature; it never reads a private key and never sees a password. Only the **outcome** of a check is ever recorded, never any secret.

Caddy handles auth via `forward_auth`: every request is checked against `helix-auth`'s `GET /auth` endpoint (valid cookie → 200, else 401). The login page is served by `GET /login` (which issues the challenge), the signed challenge is posted to `POST /login`, and `POST /logout` clears the cookie.

Why SSH-key and not PAM

The original design verified the real account **password live via PAM**. That model was **abandoned** because it cannot work as a non-root service on this host (captured facts, §11.4.6 — not guessing):

- This host is ALT Linux with the **tcb** shadow scheme. `/etc/tcb` is `drwx--x---` root shadow and the per-user `/etc/tcb/<user>/shadow` is **not readable by the user**.
- The `tcb_chkpwd` `setuid` helper lives in `/usr/lib/chkpwd/`, which is **permission-denied to a non-root process**.
- Therefore `pam_tcb` running as our **non-root** `systemd --user gate` can neither read the hash nor invoke the helper: `pam_authenticate` returns `PAM_AUTH_ERR` for **every** password — the wrong one *and* the correct one (both verified → `rc=7`). Non-root PAM password verification is impossible here.

- A password gate would need a **root/system** auth daemon, but the operator directive is that `sudo/root` cannot be used and that **all access must use SSH keys**.

SSH-key challenge-response needs none of that: `~/.ssh/authorized_keys` is readable by the user, `ssh-keygen -Y sign / -Y verify` are supported, so the whole flow runs **as non-root milosvasic**, with **no password and nothing stored** — and it ties directly to the same keys the real account already trusts.

3. Editor Explorer default view vs. full-capability session

The two surfaces are intentionally scoped differently — but this is a **convenience default, NOT a security boundary**:

- **Editor Explorer — defaults to the projects directory.** code-server opens `$PROJECTS_ROOT` as the workspace, so the file Explorer *defaults* to that folder for convenience. **It is NOT a security boundary** — the integrated terminal, **File > Open Folder**, tasks, the debugger, and extensions retain full real-user (milosvasic) access to the host filesystem **BY DESIGN**. There is no code-server flag that confines the process to `$PROJECTS_ROOT`.
- **Integrated terminal — full real-user capability.** The terminal is a real login shell running as the real user, so it keeps **full capability by design**: SSH-key git operations, every host binary, and the complete `.bashrc/profile` environment.

This split is deliberate and matches the operator decision (*editor file-tree scoped for focus; shell full-capability*). Isolation, if it were ever required, would need a container / VM / chroot — **out of scope per operator decision**.

Defense-in-depth (deploy stream, in progress): code-server runs with `--disable-workspace-trust`, and the `/proxy/` path is blocked at the Caddy edge.

4. Parameters (replacing CODE_SERVER_PASSWORD)

The old single-password model (`CODE_SERVER_PASSWORD`) is **retired**. The real-account SSH-key model uses:

Parameter	Meaning	Example / default
HELIX_AUTH_MODE	Authentication mode. sshkey is the only mode in this release.	sshkey
HELIX_AUTH_ACCOUNT	The real host user account each session ties to.	milosvasic
HELIX_AUTH_AUTHORIZED_KEYS	Path to that account's authorized-keys file — the verifier's trust source.	~/.ssh/authorized_keys
HELIX_AUTH_PRINCIPAL	The signer principal expected in the signature (defaults to the account name).	milosvasic
PROJECTS_ROOT	Workspace + default Explorer folder (convenience view, not a jail; existing PROJECTS).	<i>(your projects directory)</i>

- **There is no password parameter.** Login is a signed challenge, verified against HELIX_AUTH_AUTHORIZED_KEYS (see §2) — never a stored or configured secret.
- .env.example carries **placeholders only** (never a real secret, §11.4.10); CODE_SERVER_PASSWORD is removed from the parameter set.

5. Security posture

- **Auth fails CLOSED.** This is the non-negotiable security invariant: if the helix-auth gate is down, errored, or unreachable, Caddy **denies** access — it never proxies to code-server and never “bypasses” the check on failure. No gate → no access.
- **Challenge is server-issued + short-lived.** Each GET /login issues a fresh signed nonce bound to a short TTL, so a

signature captured from one login cannot be replayed against another; an expired or unrecognized challenge is rejected.

- **Session cookie hardened.** The cookie is **HMAC-signed**, **HttpOnly**, **Secure**, and **SameSite (Lax)**, with a session TTL (sliding). The HMAC secret lives in a `0600` file (regenerated if absent). Corrupting the secret **invalidates** existing sessions — it does not bypass auth.
- **Login is rate-limited.** The login endpoint is rate-limited to resist brute-force / flooding; the limiter fails closed.
- **Least privilege + loopback only, NON-root.** `helix-auth` binds loopback-only and runs **as the non-root real user**. It reads only the account's public `authorized_keys` to verify signatures — never a private key, never a password — so **no root daemon is required** (the exact reason the PAM model was dropped, see §2 “Why SSH-key and not PAM”).
- **TLS enforced.** Auth only happens behind the Caddy HTTPS edge (see [docs/guides/TLS.md](#)).

Not yet validated (§11.4.6 / §11.4 anti-bluff). The `helix-auth` gate, the challenge/verify flow, fail-closed behavior, cookie hardening, and rate limiting are **designed and being built**, and their tests/evidence are being produced by other work streams — treat the security claims above as *intended*, not yet proven, until the feature ledger records a PASS.

5.1. Residual risk — loopback reachability, and how to close it

The risk (honest, §11.4.6). The host-native code-server runs with `--auth none` and binds `127.0.0.1:8080`. Authentication is enforced **upstream** by the Caddy `forward_auth` gate, **not** on the loopback socket. So the loopback socket has **no per-UID access control**: any local process or other user in the host network namespace can connect to `127.0.0.1:8080` directly and get an interactive shell **as the account** — bypassing the Caddy gate entirely. (The strongest fix — a unix socket bind-mounted into the rootless Caddy container — was assessed and **deferred** as fragile on this two-lifecycle topology; see the note in `deploy/compose.codeserver.yml`. This is the same residual risk documented in the RESIDUAL RISK block of `deploy/systemd/helix-code-server.service`.)

The mitigation. A **UID-scoped loopback OUTPUT firewall rule** that DROPS connections to `127.0.0.1:8080` from any UID **other than the account**. Only the account reaches code-server; rootless Caddy connects **as the account** (via `host.containers.internal` → host loopback), so the gate path stays allowed. Rootless-Podman pods sit in their own network namespace and never reach host loopback unless host-networked.

How to run it. scripts/harden-loopback.sh installs the rule (nftables preferred, iptables fallback). Because modifying the firewall needs root and this project **never uses sudo/root itself**, the operator applies it via their own root path. It reads the account + port from deploy/.env — nothing is hard-coded.

```
# 1. Inspect the current state (read-only, NO root) – prints the
    exact rule it
#    would add:
scripts/harden-loopback.sh --check

# 2. Apply it as root, via your own root path (su / sudo / doas /
    root login):
su - -c 'HELIX_AUTH_ACCOUNT=milosvasic /abs/path/to/scripts/
    harden-loopback.sh --apply'

# 3. Remove it later (also root):
su - -c 'HELIX_AUTH_ACCOUNT=milosvasic /abs/path/to/scripts/
    harden-loopback.sh --remove'
```

The rule installed (iptables form, exactly as documented in the systemd unit):

```
iptables -A OUTPUT -o lo -p tcp --dport 8080 -m owner ! --uid-owner <account-uid> -j DROP
```

nft/iptables rules are **runtime state** and do not survive a reboot on their own — persist them via your distro’s ruleset-save mechanism or a boot unit after applying. This is **defence-in-depth** on top of the fail-closed Caddy gate, not a replacement for it. Full details: <docs/scripts/harden-loopback.md>.

6. Operator prerequisites

The real-account model assumes the host is already set up for the target user (default milosvasic):

1. **The host user exists** and is the account you want sessions tied to (HELIX_AUTH_ACCOUNT).
2. **SSH keys are already set up** for that user in the host ~/.ssh and their public keys are present in HELIX_AUTH_AUTHORIZED_KEYS — the login verifies a signature against those keys, and the editor terminal uses the same keys for git as-is (nothing is copied into the project).
3. **code-server is installed** for that user (host-native; the intended install path is the user-level npm global install — the exact binary source is being finalized in the design’s P0 spike).
4. **systemd --user + lingering** is enabled for the user, so the host-native code-server (and auth) services survive logout/reboot (loginctl enable-linger <user>).

5. `HELIX_AUTH_MODE=sshkey`, `HELIX_AUTH_ACCOUNT=<user>`, `HELIX_AUTH_AUTHORIZED_KEYS`, `HELIX_AUTH_PRINCIPAL`, and `PROJECTS_ROOT` are set in the deploy env.
-

7. Troubleshooting

Login is rejected even though I signed the challenge. - Sign with a key whose **public key is in `HELIX_AUTH_AUTHORIZED_KEYS`** for the account, using the exact namespace: `-n helixcode-login`. A signature from a key that is not authorized, or under a different namespace, is rejected. - Sign the challenge **shown on the current login page** — challenges are short-lived and single-issue, so a signature over an old/expired challenge is rejected. Reload the page to get a fresh challenge and re-sign. - Confirm the **principal** you submit matches `HELIX_AUTH_PRINCIPAL` (defaults to the account name). - Because auth **fails closed**, a rejected login when everything looks correct usually means the helix-auth gate is down or unreachable — check that the gate is running and reachable on loopback.

ssh-keygen -Y sign fails locally. - Point `-f` at a private key you actually hold (e.g. `~/.ssh/id_ed25519`); the signing runs **on your machine**, and the private key never leaves it.

Git over SSH doesn't use my key from the terminal. - The terminal must be a **login shell** running as `HELIX_AUTH_ACCOUNT`; confirm the session opened a bash `-l` shell and that `~/.ssh` belongs to that user with sane permissions (700 on `~/.ssh`, 600 on private keys). - Verify from the terminal with a non-mutating probe, e.g. `git ls-remote git@github.com:vasic-digital/Code-Server.git`.

A `.bashrc` export / `PATH` entry is missing in the terminal. - Confirm the terminal is a login shell (echo `$0` shows `-bash` / a leading `-`), since `~/.bashrc` + profile are sourced by the login shell. A non-login shell won't source the full profile.

The Explorer only shows my projects folder — where's the rest of the host? - The Explorer **defaults** to `$PROJECTS_ROOT` for convenience; this is **not a jail**. Use **File > Open Folder** to open any path, or the **integrated terminal** (full real-user capability) to reach anything on the host as `milosvasic`. Real isolation would require a container / VM / `chroot` (out of scope per operator decision).

Honest boundary (§11.4.6)

This guide documents the **intended** real-account + SSH-key challenge-response behavior from the approved pivot spec. The feature is **under construction and not yet validated**: the helix-auth gate, the challenge/verify flow, fail-closed, cookie, and rate-limit behaviors are covered by tests/evidence that **other work streams are still building**. Do not treat any capability here as proven until [docs/features/Status.md](#) records its validation verdict.

Sources verified

- Pivot spec (authoritative): [docs/superpowers/specs/2026-07-01-auth-pivot-ssh-key.md](#) — verified 2026-07-01.
- Superseded design spec (PAM login part retired): [docs/superpowers/specs/2026-07-01-real-account-code-server-design.md](#) — verified 2026-07-01.
- TLS edge behavior: [docs/guides/TLS.md](#) — verified 2026-07-01.